

EXEMPLOS DE TESTES:

1. Testes Não Funcionais (Performance, Carga, Estresse)

A. Exemplo de Teste de Performance (Foco no Tempo de Resposta)

- **Cenário:** Emissão de um relatório financeiro diário por usuários de um sistema ERP.
- **Como testar:** Utilizando uma ferramenta como o JMeter, simula-se um número normal e contínuo de usuários gerando o relatório simultaneamente.
- **Objetivo:** O foco aqui não é quebrar o sistema, mas sim **avaliar o tempo de resposta** e a latência da aplicação durante uma operação rotineira. O teste busca garantir, por exemplo, que a tela carregue em menos de 2 segundos, mapeando se o processamento da query no banco de dados está gerando lentidão.

B. Exemplo de Teste de Carga (Foco no Limite Esperado)

- **Cenário:** Lançamento de um produto ou promoção de Black Friday em um e-commerce.
- **Como testar:** Configura-se uma ferramenta (como K6 ou Gatling) para disparar tráfego simulando o pico máximo de acessos previstos pela equipe de marketing (por exemplo, testando se o sistema suporta **10.000 cliques simultâneos** no botão "Comprar").
- **Objetivo:** Avaliar o **comportamento no limite esperado** do sistema. O teste identifica se, ao atingir a carga máxima projetada, algum componente da arquitetura (rede, banco de dados ou servidor) começa a atuar como um gargalo limitador antes do esperado.

C. Exemplo de Teste de Estresse (Foco na Quebra do Sistema)

- **Cenário:** Viralização inesperada de um aplicativo, gerando um volume de acessos massivamente superior à capacidade da infraestrutura atual.
 - **Como testar:** Injeta-se uma carga de usuários virtuais que aumenta progressivamente, ultrapassando intencionalmente a capacidade máxima dos servidores, até que a aplicação pare de responder ou caia (downtime).
 - **Objetivo:** Avaliar o **comportamento além do limite máximo até a quebra do sistema**. O objetivo crucial é descobrir exatamente qual é o "elo mais fraco" da arquitetura — ou seja, identificar se a quebra ocorreu por falta de memória (processamento), estouro de conexões no banco de dados ou limite de banda de rede.
-

2. Testes de Usabilidade e Acessibilidade Digital (UX)

A. Teste de Acessibilidade Visual com Leitores de Tela (Foco WCAG)

- **Cenário:** Navegação de um usuário com deficiência visual em um formulário de login ou checkout.
- **Como testar:** O testador utiliza leitores de tela (como NVDA ou VoiceOver) para navegar pela página sem a ajuda do mouse ou do monitor. Verifica-se se todas as imagens, botões e campos de texto possuem descrições de texto alternativas programadas corretamente no código.
- **Objetivo:** Aplicar o padrão WCAG (Web Content Accessibility Guidelines). Em vez de uma avaliação subjetiva, este teste traz uma regra técnica e binária (passou/falhou) : se o leitor de tela não conseguir ler o propósito de um botão de "Enviar", o sistema falha no critério de acessibilidade, garantindo que o software possa ser utilizado por pessoas com deficiências visuais.

B. Teste Heurístico de Usabilidade (Foco em Interface Intuitiva)

- **Cenário:** Avaliação do fluxo de recuperação de senha em um aplicativo.

- **Como testar:** Um analista de testes ou designer utiliza as 10 Heurísticas de Jakob Nielsen para realizar um teste heurístico (uma avaliação técnica que ocorre antes de um teste de usabilidade com usuários reais). Ele avalia, por exemplo, a heurística de "Prevenção de Erros" e "Controle e Liberdade do Usuário".
- **Objetivo:** Avaliar se a interface é intuitiva, eficiente e agradável para o usuário comum. O teste busca garantir que, caso o usuário digite um e-mail com formato inválido, a interface avise o erro imediatamente de forma clara e forneça uma opção fácil para cancelar ou corrigir a ação.

C. Teste de Acessibilidade Motora via Navegação por Teclado (Foco WCAG)

- **Cenário:** Acesso ao menu principal e submenus de um portal corporativo por um usuário com deficiência motora que não consegue utilizar um mouse.
 - **Como testar:** O testador abandona o mouse e utiliza exclusivamente a tecla "Tab" do teclado para tentar alcançar todos os links, botões e campos interativos da página.
 - **Objetivo:** Validar outra regra testável e técnica focada nas Diretrizes de Acessibilidade Web (WCAG). O roteiro de teste deve responder de forma binária: a ordem de navegação ("Tab") é lógica? Existe um contorno visual (indicador de foco) destacando onde o usuário está no momento? Se a resposta for não, o sistema falha em cumprir os requisitos legais e éticos de mercado.
-

3. Fundamentos de Segurança da Informação nos Testes

A. Teste de Validação de Input contra Injeção de Código (SQL Injection)

- **Cenário:** Campo de login ou barra de busca de produtos em um sistema web.
- **Como testar:** O testador insere comandos estruturados (como ' OR '1'='1') diretamente nos campos de texto em vez de credenciais válidas ou termos de pesquisa comuns.
- **Objetivo:** Validar se as entradas de dados estão adequadamente blindadas contra injeções de código malicioso. O teste verifica se a aplicação sanitiza os inputs ou se permite que a consulta manipulada alcance o banco de dados, o que poderia expor ou corromper as informações do sistema.

B. Teste de Proteção contra Cross-Site Scripting (XSS)

- **Cenário:** Seção de comentários de um blog, fórum ou formulário de avaliação de produtos.
- **Como testar:** Um testador submete um trecho de código JavaScript (como `<script>alert('Vulnerável')</script>`) dentro do campo de texto de comentário. Em seguida, verifica-se se o código é executado no navegador de outros usuários que acessarem aquela mesma página.
- **Objetivo:** Validar se o sistema protege os dados do usuário, garantindo que o software não execute scripts não autorizados na interface gráfica. Avaliar essa falha transforma a teoria de segurança do OWASP em uma prática essencial para proteger sessões e dados sensíveis.

C. Teste de Robustez de Autenticação e Gerenciamento de Sessão

- **Cenário:** Tela de autenticação principal de um aplicativo financeiro ou painel administrativo.
 - **Como testar:** Utilizando técnicas de DAST ou testes manuais, o avaliador realiza múltiplas tentativas de login em um curto período com senhas erradas (simulando um ataque de força bruta) para ver se a conta é bloqueada após um limite.
 - **Objetivo:** Aplicar práticas de garantia de segurança (QA voltado para Security) para validar se as autenticações do sistema são realmente robustas. Esse teste foca em garantir que os riscos cibernéticos críticos sejam mitigados na prática, como exigido pelos padrões do mercado.
-

DESAFIO FINAL (GABARITO):

Aqui estão as respostas detalhadas para cada uma das perguntas de pesquisa, baseadas nos conceitos de **qualidade**, **experiência do usuário** e **segurança** de software:

Testes Não Funcionais (Performance, Carga, Estresse)

1. Qual é a diferença prática entre avaliar o tempo de resposta em testes de performance, o comportamento no limite esperado em testes de carga e o comportamento até a quebra do sistema em testes de estresse?

- **Teste de Performance:** Foca em avaliar a "saúde" e a velocidade do sistema no dia a dia. A pergunta que ele responde é: *"Em condições normais, quão rápido a tela carrega?"* (Avalia o tempo de resposta).
- **Teste de Carga:** Foca em validar se o sistema suporta o tráfego máximo para o qual foi planejado. A pergunta é: *"No pico de acessos esperado (ex: Black Friday), o sistema continua operando bem ou fica insuportavelmente lento?"*
- **Teste de Estresse:** Foca em destruir temporariamente a aplicação de forma controlada. A pergunta é: *"Se o tráfego ultrapassar nosso limite máximo, o que quebra primeiro e como o sistema se recupera?"* (Avalia a robustez e o ponto de ruptura).

2. O que representam as métricas de performance "Throughput" e "Latência", e como elas são medidas utilizando ferramentas como JMeter, Gatling ou K6?

- **Throughput (Vazão):** Representa a capacidade do sistema de processar requisições em um determinado tempo (ex: 500 requisições por segundo). É a largura de banda efetiva.
- **Latência:** É o tempo exato que um pacote de dados leva para sair do dispositivo do usuário, chegar ao servidor, ser processado e iniciar o caminho de volta.
- **Como são medidas:** Ferramentas como JMeter criam "usuários virtuais" (threads) que disparam requisições HTTP para o servidor. Elas marcam o timestamp (carimbo de tempo) de envio e o timestamp de recebimento da resposta, calculando matematicamente a latência e o throughput médio gerando relatórios precisos.

3. Por que a identificação do gargalo de desempenho (Bottleneck) na arquitetura de um sistema (seja no banco de dados, rede ou processamento) é o objetivo principal dos testes não funcionais, e não apenas a execução da ferramenta?

Apenas saber que um sistema "está lento" não ajuda os desenvolvedores a consertá-lo. O objetivo é encontrar a causa raiz. Se o teste de carga mostra lentidão, o gargalo pode ser a CPU do servidor que chegou a 100%, a memória que estourou, o banco de dados que está travando as consultas (queries) ou a rede que não tem banda suficiente. Identificar o *bottleneck* permite focar os esforços técnicos e financeiros no local exato do problema.

Testes de Usabilidade e Acessibilidade Digital (UX)

4. O que são as 10 Heurísticas de Jakob Nielsen e como elas orientam a avaliação de uma interface para garantir que ela seja intuitiva e eficiente?

São princípios gerais de design de interação criados por Jakob Nielsen na década de 1990. Elas servem como um "checklist mental" para especialistas avaliarem interfaces. Regras como "Visibilidade do status do sistema" (o usuário deve saber o que está acontecendo) e "Prevenção de erros" (é melhor evitar que o erro ocorra do que dar uma boa mensagem de erro) orientam o teste, permitindo encontrar problemas graves de usabilidade de forma rápida antes mesmo do software chegar ao usuário final.

5. Quais são as principais diferenças, vantagens e desvantagens entre realizar um teste de usabilidade com usuários reais em comparação com um teste heurístico?

- **Teste Heurístico:** Feito por especialistas (QA ou Designers) usando as heurísticas de Nielsen.
 - *Vantagem:* Rápido, barato e encontra os problemas mais óbvios rapidamente.
 - *Desvantagem:* Pode não refletir as reais dificuldades e os caminhos inesperados que um usuário leigo tomaria.
- **Teste com Usuários Reais:** Feito observando pessoas do público-alvo utilizando o sistema.
 - *Vantagem:* Traz dados autênticos, reais e revela problemas que nenhum especialista previu.

- **Desvantagem:** Demorado, caro (exige recrutamento, incentivos) e requer laboratório ou ferramentas de gravação.

6. O que é o padrão WCAG (Web Content Accessibility Guidelines) e de que forma ele transforma a avaliação de acessibilidade em regras testáveis e binárias (passou/falhou)?

O WCAG é o manual de diretrizes internacional de acessibilidade para a web. Ele divide a acessibilidade em princípios (Perceptível, Operável, Compreensível e Robusto) e detalha "Critérios de Sucesso". Isso tira a subjetividade do teste. Por exemplo, em vez do testador achar que "a cor está ruim", ele testa a regra técnica 1.4.3 do WCAG: a taxa de contraste do texto deve ser de pelo menos 4.5:1. Se a ferramenta medir 3.0:1, o teste falhou. É matemático e exato.

7. Como ferramentas de leitores de tela, como o NVDA e o VoiceOver, devem ser utilizadas na prática para garantir que um software cumpra os requisitos legais e éticos de mercado para pessoas com deficiência visual?

Na prática, o testador deve desligar o monitor (ou vendar os olhos), guardar o mouse e navegar pela aplicação utilizando apenas o teclado (teclas Tab, Setas, Enter) ouvindo o que o leitor de tela dita. Ele deve verificar se todas as imagens possuem texto alternativo (Atributo `alt`), se a ordem de leitura faz sentido e se botões dinâmicos anunciam seu estado (ex: "Menu expandido", "Caixa de seleção marcada").

Fundamentos de Segurança da Informação nos Testes

8. O que é o projeto OWASP e por que o seu ranking "OWASP Top 10" é utilizado como a principal referência de mercado para mapear vulnerabilidades críticas em aplicações web?

OWASP (Open Worldwide Application Security Project) é uma fundação sem fins lucrativos focada em melhorar a segurança de softwares. Seu ranking "Top 10" é um documento de conscientização que lista as dez categorias de riscos de segurança mais críticos e comuns na web (como injeções e falhas de autenticação). É a principal referência porque é baseado em dados maciços coletados de empresas de cibersegurança pelo mundo todo, garantindo que as equipes de testes foquem no que os hackers reais estão explorando no momento.

9. Como a falta de validação de input de dados na camada de aplicação pode expor um sistema a injeções de código malicioso, como SQL Injection e Cross-Site Scripting (XSS)?

A falta de validação faz com que o sistema confie cegamente no que o usuário digita.

- No **SQL Injection**, se um campo de login não for sanitizado, o invasor digita um comando de banco de dados e a aplicação o executa internamente, podendo vazar senhas.
- No **XSS**, o invasor digita um código JavaScript em um campo (como nos comentários de um blog). Se o sistema não limpar esse código, ele será salvo e executado diretamente no navegador dos próximos usuários que visitarem a página, permitindo o roubo de sessões.

10. Qual é a diferença conceitual e prática entre as abordagens de avaliação de segurança SAST, DAST e o Pentest (teste de intrusão) manual?

- **SAST (Static Application Security Testing):** É um teste de "caixa branca". Ele escaneia o código fonte (texto) sem executar a aplicação, procurando vulnerabilidades (ex: senhas fixas no código, funções inseguras). Acontece cedo no desenvolvimento.
- **DAST (Dynamic Application Security Testing):** É um teste de "caixa preta". Ele ataca a aplicação rodando (em execução), tentando invadir formulários e APIs de fora para dentro, simulando um ataque automatizado.
- **Pentest Manual:** Um hacker ético utiliza ferramentas e raciocínio criativo para tentar quebrar a segurança do sistema. Diferente do DAST, o Pentest manual consegue encadear falhas complexas e entender o contexto do negócio, descobrindo vulnerabilidades que scanners automatizados jamais conseguiriam ver.